

A Note on Constraint Aggregation and Value Functions for Two-Stage Stochastic Integer Programs

Andrew C. Trapp, Oleg A. Prokopyev *

*School of Business, Worcester Polytechnic Institute, 100 Institute Rd., Worcester, MA, USA
Industrial Engineering, University of Pittsburgh, 1048 Benedum Hall, Pittsburgh, PA, USA*

Abstract: We consider a class of two-stage stochastic integer programs and their equivalent reformulation that uses the integer programming value functions in both stages. One class of solution methods in the literature is based on the idea of pre-computing and storing exact value functions, and then exploiting this information within a global branch-and-bound framework. Such methods are known to be very sensitive to the magnitude of feasible right-hand side values. In this note we propose a simple constraint-aggregation based approach that potentially alleviates this limitation.

Keywords: Stochastic programming, value function, constraint aggregation, branch-and-bound

1. Introduction

We consider a class of two-stage stochastic integer programs (SIPs) of the form:

$$\max_{x \in \mathbb{Z}_+^{n_1}} \left\{ c^\top x + \mathbb{E}_\omega Q(x, \omega) \mid Ax \leq b \right\}, \quad (\text{SIP})$$

where, given $x \in \mathbb{Z}_+^{n_1}$ and $\omega \in \Omega$, the second-stage recourse function $Q(x, \omega)$ is defined as:

$$Q(x, \omega) = \max_{y \in \mathbb{Z}_+^{n_2}} \left\{ d(w)^\top y \mid W(w)y \leq h(\omega) - Tx \right\}.$$

Random variable ω having support Ω characterizes the uncertain parameter realizations (scenarios). The number of constraints and variables is m_i and n_i , respectively, for stages $i = 1, 2$. Note that the technology matrix T is assumed to be deterministic. In the remainder of the paper we make the following additional assumptions:

A1. The random variable ω is discretely distributed with finite support.

A2. The first-stage feasibility set $X = \{x \mid x \in \mathbb{Z}_+^{n_1}, Ax \leq b\}$ is nonempty and bounded.

A3. $Q(x, \omega)$ is finite for all $x \in X$ and $\omega \in \Omega$.

*Corresponding author. Email: droleg@pitt.edu.
Phone: +1-412-624-9833.

A4. The first-stage constraint matrix A and technology matrix T as well as recourse matrices $W(\omega)$ for all $\omega \in \Omega$ are integral, i.e., $A \in \mathbb{Z}^{m_1 \times n_1}$, $T \in \mathbb{Z}^{m_2 \times n_1}$ and $W(\omega) \in \mathbb{Z}^{m_2 \times n_2}$ $\forall \omega \in \Omega$.

Assumption **A1** is justified in [11], **A2** ensures that X is finite, and **A3** establishes relatively complete recourse. Assumptions similar to **A1** – **A4** are common in the related stochastic optimization literature, see, e.g., [1–3, 8, 10, 12]. Based on **A4**, without loss of generality, let $c \in \mathbb{Z}^{n_1}$, $d(\omega) \in \mathbb{Z}^{n_2}$, $b \in \mathbb{Z}^{m_1}$ and $h(\omega) \in \mathbb{Z}^{m_2}$ $\forall \omega \in \Omega$.

In the subsequent discussion we closely follow the notation (in particular, with respect to value functions) from [8, 10]. For any $\beta_1 \in \mathbb{Z}^{m_2}$, define the first-stage value function as:

$$\psi(\beta_1) = \max \{c^\top x \mid x \in S^1(\beta_1)\}, \quad S^1(\beta_1) = \{x \in \mathbb{Z}_+^{n_1} \mid Ax \leq b, Tx \leq \beta_1\}. \quad (1)$$

For any $\beta_2 \in \mathbb{Z}^{m_2}$, define the second-stage value function for each ω as:

$$\phi_\omega(\beta_2) = \max \{d(\omega)^\top y \mid y \in S_\omega^2(\beta_2)\}, \quad S_\omega^2(\beta_2) = \{y \in \mathbb{Z}_+^{n_2} \mid W(\omega)y \leq \beta_2\}. \quad (2)$$

Define $\mathbb{B}^1 = \{\beta_1 \in \mathbb{Z}^{m_2} \mid S^1(\beta_1) \neq \emptyset\}$, that is, $\mathbb{B}^1$ is the set of vectors $\beta_1 \in \mathbb{Z}^{m_2}$ such that there exists a feasible first-stage $x \in S^1(\beta_1)$. Define $\mathbb{B}^2 \subseteq \{\beta_2 \in \mathbb{Z}^{m_2} \mid S_\omega^2(\beta_2) \neq \emptyset \forall \omega \in \Omega\}$ such that $\mathbb{B}^2 \supseteq \bigcup_{\beta_1 \in \mathbb{B}^1} \bigcup_{\omega \in \Omega} \{h(\omega) - \beta_1\}$, i.e., $\mathbb{B}^2$ contains all vectors $\beta_2 \in \mathbb{Z}^{m_2}$ such that there exists $\beta_1 \in \mathbb{B}^1$ with $\beta_2 = h(\omega) - \beta_1$. Note that **A2** and **A3** imply $\mathbb{B}^1 \neq \emptyset$ and $\mathbb{B}^2 \neq \emptyset$. We then reformulate SIP as:

$$\max_{\beta_1 \in \mathbb{B}^1} \{\psi(\beta_1) + \mathbb{E} \phi_\omega(h(\omega) - \beta_1)\}. \quad (\text{P1})$$

Theorem 1 *Let β_1^* solve (P1). Then $\hat{x} \in \operatorname{argmax} \{c^\top x \mid Ax \leq b, Tx \leq \beta_1^*, x \in \mathbb{Z}_+^{n_1}\}$ is an optimal solution to (SIP). Moreover, the optimal objective function values are equal.*

Theorem 1 (or its variations) can be found in [1, 3, 6, 8, 12]. Value function reformulation (P1), like its versions for other classes of stochastic integer programs in the related literature, can be exploited to solve the original problem (SIP), see [1, 3, 6, 8, 10, 12]. In particular, one class of solution methods, see [8, 10], is based on the following idea:

- **Phase 1:** Compute and completely store in computer memory the value functions for all (or almost all) β_1 in $\mathbb{B}^1$ and β_2 in $\mathbb{B}^2$ (these sets are assumed to be finite, see Section 3.1);
- **Phase 2:** Apply a global branch-and-bound (B&B) algorithm over $\mathbb{B}^1$ to solve (P1).

The main advantage of such an approach is its somewhat reduced sensitivity to the number of variables and scenarios in the second stage under the following condition. Let $\Gamma = \{\Gamma_1, \dots, \Gamma_{|\Gamma|}\}$, where each $\Gamma_s \subseteq \Omega$, $s \in \{1, \dots, |\Gamma|\}$. Assume that for any $\omega_1 \in \Gamma_s$ and $\omega_2 \in \Gamma_s$ we have $d(\omega_1) = d(\omega_2)$ and $W(\omega_1) = W(\omega_2)$. In other words, the scenarios can be divided into a set of groups Γ sharing the same d and W (but with no restrictions on the right-hand side h). Observe that for all scenarios in Γ_s , $s \in \{1, \dots, |\Gamma|\}$, value functions (2) over $\mathbb{B}^2$ coincide. Thus, if the number of groups in Γ is relatively small, then the number of value functions (2) that need to be computed and stored during Phase 1 is also small, i.e., $|\Gamma|$.

In Phase 2, at each iteration k of the global B&B algorithm (see [8, 10] for detailed discussion and algorithmic pseudo-code for each phase) we consider a single hyper-rectangle

$\mathcal{H}^k = [\ell^k, u^k] \subseteq \mathbb{B}^1$ from a list of unfathomed hyper-rectangles with a subproblem of the form

$$f^k = \max \{ \psi(\beta_1) + \mathbb{E}\phi_\omega(h(\omega) - \beta_1) \mid \beta_1 \in \mathcal{H}^k \cap \mathbb{Z}^{m_2} \}. \quad (3)$$

Due to the nondecreasing property of the integer programming (IP) value function (see Proposition 1 in Section 2) the corresponding upper bound $v^k \geq f^k$ is given by

$$v^k = \psi(u^k) + \mathbb{E}\phi_\omega(h(\omega) - \ell^k), \quad (4)$$

A hyper-rectangle $\mathcal{H}^k$ is fathomed if $L \geq v^k$, where L is a global lower bound, e.g., the best feasible solution available at iteration k . The *key observation*, which makes this two-phase framework competitive for some classes of (SIP), is that (4) can be *quickly evaluated* (in particular, the second-stage term $\mathbb{E}\phi_\omega(\cdot)$ with, possibly, a very large number of scenarios ω) due to the fact that $\psi(\cdot)$ and $\phi_\omega(\cdot)$ are readily available after Phase 1.

In [8] the authors report encouraging computational results for a class of (SIP), where the recourse matrix W and the second-stage objective d are deterministic, leaving h as the lone stochastic component of the problem, i.e., $|\Gamma| = 1$. The algorithms in [8] are capable of solving instances of (SIP) with very large extensive forms (e.g., around 2 million rows, 140 million variables), which are among the largest instances (with respect to the extensive form sizes) in the stochastic optimization literature. Results of [8] are generalized for two-stage stochastic quadratic integer programs in [10]. Interestingly, the sizes of instances solved in [10] have similar orders of magnitude as those of [8], leading us to conclude that this framework may alleviate some of the difficulties arising due to nonlinearity of the objective functions, at least for the classes of SIPs considered in [10]. Some intuition behind this observation is that the objective function nonlinearities are taken into account mainly in Phase 1 of the approach.

However, the above advantages come at the price of high sensitivity to both the number of constraints in each stage as well as the magnitude of right-hand sides because of value function storage requirements of $O(|\mathbb{B}^1| + |\Gamma| \cdot |\mathbb{B}^2|)$. For example, if $0 \leq h(\omega) \leq \bar{h}$ for all $\omega \in \Omega$ and $\bar{h} \in \mathbb{Z}^{m_2}$, then $|\mathbb{B}^2| = O(\bar{h}^{m_2})$. Thus, the maximum right-hand side of test instances reported in [8] is rather small, i.e., $\bar{h} \leq 10$, to accommodate the limited memory availability on a regular PC with 2GB RAM. The maximum right-hand side in [10] is somewhat larger ($\bar{h}$ is varied from 10 to 300 for different classes of test instances) due to the availability of a high-performance shared-memory machine.

The overall goal of this note is to describe an additional, and relatively simple, approach to reduce memory requirements for this class of solution methods. In particular, instead of exact value functions we compute and store their approximations (specifically, their upper bounds) constructed using constraint aggregation reasoning. The main advantage of the proposed concept is its reduced memory requirements, under some mild conditions. These approximations are subsequently exploited to compute an appropriate upper bound for f^k by a slight modification of equation (4).

The remainder of this note is organized as follows. In Section 2, given an integer programming value function defined over a finite set of right-hand sides, we study a class of value functions with a reduced number of constraints, that serve as its upper-bounding approximation. In Section 3 we provide a computational proof-of-concept for our approach, solving the same class of (SIP) as in [8]. In particular, we first describe a bounding scheme for the global B&B algorithm in Section 3.1. We then develop a dynamic programming-based algorithm

for computing the proposed value functions in Section 3.2. In Section 3.3 we report computational results for test instances that cannot be handled by the approaches of [8, 12]. Finally, Section 4 concludes the discussion and highlights possible directions for future research work.

2. Constraint Aggregation and Integer Programming Value Function

Background on IP Value Function. Given $\gamma \in \mathbb{Z}^n$ and $G \in \mathbb{Z}^{m \times n}$, the IP value function $z(\cdot) : \mathbb{Z}^m \mapsto \mathbb{Z}$ is defined as (see, e.g., [9]):

$$z(\beta) = \max \{ \gamma^\top x \mid x \in S(\beta) \}, \quad S(\beta) = \{ x \in \mathbb{Z}_+^n \mid Gx \leq \beta \},$$

where $S(\beta)$ represents the feasible region of $z(\beta)$ for any $\beta \in \mathbb{Z}^m$. We impose integrality restrictions for γ , G and β parallel to **A4**. Denote the set containing all optimal solutions to $z(\beta)$ by $\text{opt}(\beta) = \text{argmax}_x \{ \gamma^\top x \mid x \in S(\beta) \}$. Define $z(\beta) = -\infty$ if $S(\beta) = \emptyset$. Let g_j and γ_j be the j^{th} column of G and the j^{th} coefficient of γ , respectively. We use 0 for a vector of zeros of conformable dimension. Next, we review some standard results related to the properties of the IP value function.

Proposition 1 *Properties of the integer programming value function include:*

- (i) $z(g_j) \geq \gamma_j$ for $j = 1, \dots, n$.
- (ii) $z(\cdot)$ is nondecreasing over $\mathbb{Z}^m$.
- (iii) $z(\cdot)$ is superadditive, that is $z(\beta_1) + z(\beta_2) \leq z(\beta_1 + \beta_2)$ for all β_1, β_2 .

Proposition 2 [*Integer Complementary Slackness (ICS)*] *If $\hat{x} \in \text{opt}(\beta)$ for $\beta \in \mathbb{Z}^m$, then for all $x \in \mathbb{Z}_+^n$ such that $x \leq \hat{x}$, $z(Gx) = \gamma^\top x$ and $z(Gx) + z(\beta - Gx) = z(Gx) + z(G(\hat{x} - x)) = z(\beta)$.*

Proofs of Propositions 1 and 2 as well as other results related to the IP value function can be found in [9].

Constraint Aggregation. Consider an aggregation of the constraints in $S(\beta)$ obtained by left-multiplying $G \in \mathbb{Z}^{m \times n}$ with nonnegative matrix $\mathcal{A} \in \mathbb{Z}_+^{\bar{m} \times m}$, $\bar{m} < m$, that yields

$$S_{\mathcal{A}}(\tau) = \{ x \in \mathbb{Z}_+^n \mid \mathcal{A}Gx \leq \tau \}, \quad \tau \in \mathbb{Z}^{\bar{m}},$$

which is referred to as an *aggregated feasibility set*. Simply speaking, each constraint in the constraint set of $S_{\mathcal{A}}(\cdot)$ is obtained by multiplying (according to the values in $\mathcal{A}$) and summing constraints from $S(\cdot)$. Then we define the *aggregate* value function $z_{\mathcal{A}}(\cdot) : \mathbb{Z}^{\bar{m}} \mapsto \mathbb{Z}$ given by:

$$z_{\mathcal{A}}(\tau) = \max_{x \in \mathbb{Z}_+^n} \{ \gamma^\top x \mid \mathcal{A}Gx \leq \tau \}, \quad \tau \in \mathbb{Z}^{\bar{m}}. \quad (5)$$

Let $\text{opt}_{\mathcal{A}}(\tau) = \text{argmax}_x \{ \gamma^\top x \mid x \in S_{\mathcal{A}}(\tau) \}$. By nonnegativity of $\mathcal{A}$, we have:

$$S(\beta) \subseteq S_{\mathcal{A}}(\mathcal{A}\beta) \quad \text{and} \quad z(\beta) \leq z_{\mathcal{A}}(\mathcal{A}\beta), \quad (6)$$

so $z_{\mathcal{A}}(\mathcal{A}\beta)$ is a valid upper bound for $z(\beta)$. Thus, $z_{\mathcal{A}}(\cdot)$ can be used instead of $z(\cdot)$ in the global B&B framework for solving (P1) by a slight modification of (4). More importantly,

if $\bar{m}$ is sufficiently small, then the memory requirement for storing the value functions is potentially reduced (note that the maximum right-hand side in $S_{\mathcal{A}}$ may increase as a result of $\mathcal{A}$). Unfortunately, our preliminary computational experiments indicate that the benefits of this simple technique are at best limited (see additional discussion in Section 3.2). We should also mention that the idea of constraint aggregation is not new in the literature, see, e.g., work on surrogate duality in [5]. Next, we describe another approach based on a somewhat similar idea, which turns out to be more promising.

Mapped Value Function. Consider a subset of right-hand sides $\mathbb{B} \subseteq \mathbb{Z}^m$. Define

$$\mathbb{B}_{\mathcal{A}} = \left\{ \tau \in \mathbb{Z}^{\bar{m}} \mid \exists \beta, \tilde{\beta} \in \mathbb{B} \text{ such that } \mathcal{A}\beta \leq \tau \leq \mathcal{A}\tilde{\beta} \right\},$$

which also implies that

$$\mathbb{B}_{\mathcal{A}} \supseteq \left\{ \tau \in \mathbb{Z}^{\bar{m}} \mid \exists \beta \in \mathbb{B} \text{ such that } \tau = \mathcal{A}\beta \right\}.$$

In the remainder of this section we make the following additional assumptions for $\mathbb{B}$:

A5. $\mathbb{B}$ is finite, and for any $\beta \in \mathbb{B}$, $S(\beta) \neq \emptyset$ and $z(\beta) < +\infty$.

A6. For any $\beta \in \mathbb{B}$, $G\hat{x} \in \mathbb{B}$ for all $\hat{x} \in \text{opt}(\beta)$.

Define the *mapped* value function $z_a(\cdot) : \mathbb{B}_{\mathcal{A}} \mapsto \mathbb{Z}$ as:

$$z_a(\tau) = \max_{\beta \in \mathbb{B} : \mathcal{A}\beta \leq \tau} z(\beta), \quad \tau \in \mathbb{B}_{\mathcal{A}}. \quad (7)$$

Observe that if $\mathbb{B} = \mathbb{Z}^m$, then $z_a(\cdot)$ and $z_{\mathcal{A}}(\cdot)$ coincide. However, in the considered context $\mathbb{B}$ needs to be finite because we assume complete computation and storage of value functions for the first and second stages in (P1) (hence the rationale for **A5**). The second part of assumption **A5** is due to **A2** and **A3**. Assumption **A6** is technical; recall also that $z(G\hat{x}) = z(\beta)$ for any $\hat{x} \in \text{opt}(\beta)$.

The mapped value function $z_a(\tau)$ can be rewritten as the following parameterized IP:

$$z_a(\tau) = \max_{x \in \mathbb{Z}_+^n, \beta \in \mathbb{B}} \left\{ \gamma^\top x \mid Gx \leq \beta, \mathcal{A}\beta \leq \tau \right\}, \quad \tau \in \mathbb{B}_{\mathcal{A}}. \quad (8)$$

As before, define $\text{opt}_a(\tau) = \arg\max_x \{ \gamma^\top x \mid \exists \beta \in \mathbb{B} \text{ such that } \mathcal{A}\beta \leq \tau \text{ and } x \in S(\beta) \} \subseteq \mathbb{Z}_+^n$ and $\widetilde{\text{opt}}_a(\tau) = \arg\max_{\beta} \{ z(\beta) \mid \beta \in \mathbb{B}, \mathcal{A}\beta \leq \tau \} \subseteq \mathbb{B}$, i.e., the sets of the respective optimal solutions. Similarly, denote the corresponding sets of feasible vectors as:

$$S_a(\tau) = \{ x \in \mathbb{Z}_+^n \mid \exists \beta \in \mathbb{B} \text{ for which } Gx \leq \beta \text{ and } \mathcal{A}\beta \leq \tau \} \quad \text{and} \quad \tilde{S}_a(\tau) = \{ \beta \in \mathbb{B} \mid \mathcal{A}\beta \leq \tau \},$$

where $S_a(\tau) \subseteq \mathbb{Z}_+^n$ and $\tilde{S}_a(\tau) \subseteq \mathbb{B}$. Next, we establish that $z_a(\cdot)$ maintains some of the attractive properties of $z(\cdot)$, e.g., superadditivity and ICS, and, moreover, is a better bound for $z(\cdot)$ than $z_{\mathcal{A}}(\cdot)$.

Proposition 3 $z_a(\cdot)$ is nondecreasing and superadditive over $\mathbb{B}_{\mathcal{A}}$.

Proof. The nondecreasing property of $z_a(\cdot)$ follows directly from the nondecreasing property of $z(\cdot)$. Regarding superadditivity, consider $\tau_1, \tau_2 \in \mathbb{B}_{\mathcal{A}}$ and respective $\beta_1 \in \widetilde{\text{opt}}_a(\tau_1)$ and $\beta_2 \in \widetilde{\text{opt}}_a(\tau_2)$. Observe that due to nonnegativity of $\mathcal{A}$ we have that $(\beta_1 + \beta_2) \in \tilde{S}_a(\tau_1 + \tau_2)$. Then we complete the proof using superadditivity of $z(\cdot)$ as follows:

$$\begin{aligned} z_a(\tau_1) + z_a(\tau_2) &= z(\beta_1) + z(\beta_2) \leq z(\beta_1 + \beta_2) \\ &\leq \max \{ z(\beta_1 + \beta_2) \mid \beta_1 + \beta_2 \in \mathbb{B}, \mathcal{A}(\beta_1 + \beta_2) \leq \tau_1 + \tau_2 \} = z_a(\tau_1 + \tau_2). \quad \blacksquare \end{aligned}$$

Proposition 4 $S_a(\tau) \subseteq S_{\mathcal{A}}(\tau)$.

Proof. Let $x \in S_a(\tau)$. Then by nonnegativity of $\mathcal{A}$ and x , $\mathcal{A}Gx \leq \mathcal{A}\beta \leq \tau$, implying $x \in S_{\mathcal{A}}(\tau)$. ■

Proposition 5 Let $\tau \in \mathbb{B}_{\mathcal{A}}$. Then

- (i) $z_a(\tau) \leq z_{\mathcal{A}}(\tau)$;
- (ii) $z_a(\tau) < z_{\mathcal{A}}(\tau)$ iff $\exists \hat{\beta} \in \mathbb{Z}^m \setminus \mathbb{B}$ such that $\mathcal{A}\hat{\beta} \leq \tau$ and $z_a(\tau) < z_{\mathcal{A}}(\mathcal{A}\hat{\beta}) = z(\hat{\beta})$;
- (iii) $z_a(\tau) = z_{\mathcal{A}}(\tau)$ iff $\exists \hat{x} \in \text{opt}_{\mathcal{A}}(\tau)$ such that $G\hat{x} \in \mathbb{B}$.

Proof.

- (i) Follows from Proposition 4.
- (ii) $\implies$ Suppose $z_a(\tau) < z_{\mathcal{A}}(\tau)$ and $\hat{x} \in \text{opt}_{\mathcal{A}}(\tau)$. Let $\hat{\beta} = G\hat{x}$. Then $z(\hat{\beta}) \geq \hat{x}^\top \gamma = z_{\mathcal{A}}(\tau)$. Therefore, $z(\hat{\beta}) = z_{\mathcal{A}}(\mathcal{A}\hat{\beta}) = z_{\mathcal{A}}(\tau)$ by Proposition 1 (ii) and (6). Recall that $G\hat{x} = \hat{\beta}$ and $\mathcal{A}G\hat{x} = \mathcal{A}\hat{\beta} \leq \tau$. If $\hat{\beta} \in \mathbb{B}$ then $z_a(\tau) \geq \gamma^\top \hat{x} = z_{\mathcal{A}}(\tau)$, which is a contradiction.
 $\Leftarrow$ Assume $\exists \hat{\beta} \in \mathbb{Z}^m \setminus \mathbb{B}$ such that $\mathcal{A}\hat{\beta} \leq \tau$ and $z_a(\tau) < z_{\mathcal{A}}(\mathcal{A}\hat{\beta}) = z(\hat{\beta})$. Because $\mathcal{A}\hat{\beta} \leq \tau$, $z_{\mathcal{A}}(\mathcal{A}\hat{\beta}) \leq z_{\mathcal{A}}(\tau)$ by the nondecreasing property. Therefore, $z_a(\tau) < z_{\mathcal{A}}(\mathcal{A}\hat{\beta}) \leq z_{\mathcal{A}}(\tau)$.
- (iii) $\implies$ Suppose $z_a(\tau) = z_{\mathcal{A}}(\tau)$. Let $\hat{x} \in \text{opt}_a(\tau)$. From Proposition 4 it also follows that $\hat{x} \in \text{opt}_{\mathcal{A}}(\tau)$. By the definition of $z_a(\tau)$ in (7) there exists $\beta \in \mathbb{B}$ such that $z(\beta) = z_a(\tau)$. Thus, $\hat{x} \in \text{opt}(\beta)$ and by assumption **A6** we conclude that $G\hat{x} \in \mathbb{B}$.
 $\Leftarrow$ Assume $\exists \hat{x} \in \text{opt}_{\mathcal{A}}(\tau)$ such that $G\hat{x} \in \mathbb{B}$. Let $\hat{\beta} = G\hat{x}$. Note that $\mathcal{A}\hat{\beta} \leq \tau$. Thus, $\hat{x} \in S_a(\tau)$ and the result follows from statement (i) of the proposition. ■

We illustrate the above proposition with the example below.

Example 1 Consider the following value function:

$$z(\beta) = \max_{x \in \mathbb{Z}_+^2} \{x_1 + 14x_2 \mid x_1 + x_2 \leq \beta_1, 2x_1 + 3x_2 \leq \beta_2\}$$

Let $\mathbb{B} = \{[0, 5] \times [0, 5]\} \cap \mathbb{Z}_+^2$ and $\mathcal{A} = (1, 1)$. Then $\mathbb{B}_{\mathcal{A}} = [0, 10] \cap \mathbb{Z}_+^1$ and

$$z_{\mathcal{A}}(\tau) = \max_{x \in \mathbb{Z}_+^2} \{x_1 + 14x_2 \mid 3x_1 + 4x_2 \leq \tau\}, \quad \tau \in \mathbb{B}_{\mathcal{A}}$$

- Let $\tau = 8$. Observe that $z_{\mathcal{A}}(8) = 28$ and $\hat{x} = (0, 2)^\top \in \text{opt}_{\mathcal{A}}(8)$. On the other hand,

$$z_a(8) = \max \{z((0, 5)^\top), z((1, 5)^\top), z((2, 5)^\top), z((3, 5)^\top), z((0, 4)^\top), \dots, z((5, 3)^\top)\} = 15$$

and $\tilde{x} = (1, 1)^\top \in \text{opt}_a(8)$. Note that conditions of Proposition 5(ii) are satisfied as $\hat{\beta} = (2, 6)^\top \notin \mathbb{B}$.

- Let $\tau = 5$. Then $z_{\mathcal{A}}(5) = z_a(5) = 14$ with $\hat{x} = (0, 1)^\top \in \text{opt}_{\mathcal{A}}(5)$ and $G\hat{x} = (1, 3)^\top \in \mathbb{B}$.

• Re-define $\mathbb{B} = (\{[0, 1] \times [0, 4]\} \cap \mathbb{Z}_+^2) \setminus (1, 3)^\top$ and $\mathbb{B}_{\mathcal{A}} = [0, 5] \cap \mathbb{Z}_+^1$. Note that assumption **A6** does not hold. However, it is still the case that $z_{\mathcal{A}}(5) = z_a(5) = 14$ because $(1, 4)^\top \in \mathbb{B}$ and $z((1, 4)^\top) = 14$. However, for $\hat{x} = (0, 1)^\top \in \text{opt}_{\mathcal{A}}(5)$ we have that $G\hat{x} = (1, 3)^\top \notin \mathbb{B}$ and Proposition 5(iii) does not hold.

Proposition 2 (ICS) is generalized for $z_a(\cdot)$ in a straightforward fashion as follows:

Proposition 6 *Let $\tau \in \mathbb{B}_A$ and $\hat{x} \in \text{opt}_a(\tau)$. Then $\forall x \in \mathbb{Z}_+^n$ such that $x \leq \hat{x}$, $Gx \in \mathbb{B}$ and $G(\hat{x} - x) \in \mathbb{B}$,*

$$z_a(\mathcal{A}Gx) = \gamma^\top x \quad \text{and} \quad z_a(\mathcal{A}Gx) + z_a(\mathcal{A}G(R\hat{x} - x)) = z_a(\tau).$$

Proof. The proof of this result is a direct extension of the proof of Proposition 2 (see [9]). Due to the nondecreasing property of $z_a(\cdot)$ we conclude that $z_a(\mathcal{A}G\hat{x}) = z_a(\tau)$. Consider any appropriate $x \in \mathbb{Z}_+^n$ such that $x \leq \hat{x}$. Note that $z_a(\mathcal{A}Gx) \geq \gamma^\top x$. Thus, suppose that $z_a(\mathcal{A}Gx) > \gamma^\top x$. Then

$$z_a(\mathcal{A}Gx) + z_a(\mathcal{A}G(\hat{x} - x)) > \gamma^\top x + z(G(\hat{x} - x)) \geq \gamma^\top x + \gamma^\top(\hat{x} - x) = z_a(\tau),$$

which contradicts superadditivity of $z_a(\cdot)$, and the first equality follows. The second equality can be shown in a similar manner. ■

3. Proof of Concept

3.1 Two-Stage IPs with Stochastic Right-Hand Sides

As a proof of concept of the proposed approach we demonstrate its application for the same class of (SIP) considered in [8] and referred to as the *two-stage IPs with stochastic right-hand sides* [8, 10]. Specifically, we assume that matrix W and the second-stage objective d are deterministic, which implies that h is the lone stochastic component of (SIP). Thus, $|\Gamma| = 1$ and value functions in (2) coincide, i.e., $\phi(\cdot) \equiv \phi_\omega(\cdot)$. Following [8, 10] and without loss of generality, we assume that the set of the first-stage constraints $Ax \leq b$ is incorporated into T , W and $h(\omega)$, so that $m = m_1 + m_2$.

We require $|\mathbb{B}^1|$ and $|\mathbb{B}^2|$ to be finite. Given a set of distinct aggregation matrices $\mathcal{A}^1, \dots, \mathcal{A}^P$, $P \in \mathbb{Z}_+$ and $P \geq 1$, we compute mapped value functions and the memory storage requirement is bounded by $O(|\mathbb{B}_{\mathcal{A}^1}^1| + |\mathbb{B}_{\mathcal{A}^1}^2| + \dots + |\mathbb{B}_{\mathcal{A}^P}^1| + |\mathbb{B}_{\mathcal{A}^P}^2|)$. Next, we apply the two-phase solution framework described in Section 1 with the following modifications:

- **Phase 1:** Instead of exact value functions $\psi(\cdot)$ and $\phi(\cdot)$, we compute and store their upper-bounding approximations $\psi_a^p(\cdot)$ and $\phi_a^p(\cdot)$ as defined in (7) for every $p \in \{1, \dots, P\}$.
- **Phase 2:** Within a global B&B algorithm for solving (P1), the upper bound (4) is replaced by

$$v_a^k = \min_{p \in \{1, \dots, P\}} \{\psi_a^p(\mathcal{A}^p u^k)\} + \mathbb{E}_\omega \left[\min_{p \in \{1, \dots, P\}} \{\phi_a^p(\mathcal{A}^p h(\omega) - \mathcal{A}^p \ell^k)\} \right], \quad (9)$$

which is valid by Proposition 5.

Also, recall that by assumption **A4** all data (i.e., matrices A , T , W as well as d , b and $h(\omega)$) are integral. Thus, within the B&B framework we can require u^k and ℓ^k to be integral for each hyper-rectangle $\mathcal{H}^k = [\ell^k, u^k] \subseteq \mathbb{B}^1$. To ensure convergence of the approach, we compute exactly the value of $\mu^k = \psi(\ell^k) + \mathbb{E}\phi_\omega(h(\omega) - \ell^k)$ whenever $\ell^k = u^k$, i.e., $\mathcal{H}^k$ is a singleton.

Consequently, μ^k is used to update a global lower bound L . Therefore, due to finiteness of $|\mathbf{B}^1|$ the algorithm terminates after a finite number of steps with an optimal solution.

In our computational study we further assume that matrices T and W are nonnegative. The nonnegativity restriction substantially simplifies algorithmic implementations and is also present in the computational studies of other closely related works [8, 10, 12]. Then, based on **A4** and without loss of generality we let $c \in \mathbb{Z}_+^{n_1}$, $d \in \mathbb{Z}_+^{n_2}$ and $h(\omega) \in \mathbb{Z}_+^{m_2} \forall \omega \in \Omega$. Therefore, the considered class of (SIP) can be viewed as *two-stage multidimensional knapsack problems with random budgets*.

The nonnegativity and integrality restrictions ensure the finiteness of $\mathbf{B}^1$. As we require finiteness for $\mathbf{B}^2$, this assumption can be justified, e.g., by the finiteness of $\{\cup_{\beta_1 \in \mathbf{B}_1} \cup_{\omega \in \Omega} \{h(\omega) - \beta_1\}\}$. Thus, in our computational study, in light of the nonnegativity of T and W and without loss of generality, we assume that $\mathbf{B}^1 = \prod_{i=1}^m [0, b_i] \cap \mathbb{Z}^m$ and $\mathbf{B}^2 = \prod_{i=1}^m [0, \bar{h}_i] \cap \mathbb{Z}^m$, where $\bar{h}_i = \max_{\omega \in \Omega} h_i(\omega)$, $\forall i = 1, \dots, m$ and $b_i \leq \bar{h}_i$ for all $i = 1, \dots, m$. Given an aggregation matrix $\mathcal{A}^p$, we construct $\mathbf{B}_{\mathcal{A}^p}^1 = \prod_{i=1}^m [0, (\mathcal{A}^p b_i)]$ and $\mathbf{B}_{\mathcal{A}^p}^2 = \prod_{i=1}^m [0, (\mathcal{A}^p \bar{h})_i]$ for all $p \in \{1, \dots, P\}$. The initial lower bound in the B&B algorithm is computed as $L = \max \{\psi(0) + \mathbb{E}\phi_\omega(h(\omega)), \psi(b) + \mathbb{E}\phi_\omega(h(\omega) - b)\}$.

3.2 Computing Mapped Value Function

For ease of exposition in this section, we use the value function notation of Section 2 to represent operations on both stages (i.e., vector γ is used to represent both c and d , etc.), applying in identical fashion the corresponding operations to the first- and second-stage problems.

There are two types of algorithms in [8, 10] for computing $z(\cdot)$. In particular, the first one (see Section 4.1 in [8] and Section 6.1 in [10]) is based on the ICS property and can handle problems with general G and γ . It can be generalized in a straightforward manner using Proposition 6, which extends ICS for our setting. The other approach (see Section 4.2 in [8] and Section 6.2 in [10]) is based on dynamic programming (DP) ideas. As reported in [8, 10], the latter method is more efficient; however, it requires nonnegativity of G and γ . Next, we describe such a DP-based algorithm to compute $z_a(\cdot)$.

Given $b \in \mathbb{Z}_+^m$, let $\mathbf{B} = \{[0, b_1] \times [0, b_2] \times \dots \times [0, b_m]\} \cap \mathbb{Z}^m$. Assume also that $g_j \in \mathbf{B}$ for all $j \in \{1, \dots, n\}$. If it is not the case, then $x_j = 0$ for all $x \in S(\beta)$ and $\beta \in \mathbf{B}$ (due to nonnegativity of G). Thus, the corresponding column and component can simply be removed from G and γ , respectively.

We focus on an aggregation matrix $\mathcal{A} \in \mathbb{Z}_+^{\bar{m} \times m}$, which can be viewed (allowing for possible column permutation) as an $(\bar{m} - 1) \times (\bar{m} - 1)$ identity matrix augmented with a single final row of all zeroes, and subsequently augmented with $(m - \bar{m} + 1)$ columns in $\mathbb{Z}^m$, containing all zeroes except for a one in the last dimension. Left-multiplication by such a matrix retains $(\bar{m} - 1)$ of the original rows, while aggregating the remaining $(m - \bar{m} + 1)$ into a single aggregate row, which without loss of generality is taken to be the final row. Given such $\mathcal{A}$, $\mathbf{B}_{\mathcal{A}} = \{[0, b_1] \times [0, b_2] \times \dots \times [0, b_{\bar{m}-1}] \times [0, b_{\bar{m}} + \dots + b_m]\}$.

The choice of $\mathcal{A}$ is motivated mainly by its simplicity, as it corresponds to a constraint aggregation obtained by summation of $(m - \bar{m} + 1)$ constraints into a single one. However, the considered type of $\mathcal{A}$ has two additional advantages. First, the increase of the maximum possible right-hand side of dimension $\bar{m}$ in $\mathbf{B}_{\mathcal{A}}$ is rather moderate, which ensures decreased

memory storage requirements. Second, the obtained mapping from $\mathbb{B}$ to $\mathbb{B}_{\mathcal{A}}$ is surjective, which is demonstrated next.

Lemma 1 *The transformation $\mathcal{T}(\cdot) : \mathbb{B} \mapsto \mathbb{B}_{\mathcal{A}}$ defined by $\mathcal{T}(\beta) = \mathcal{A}\beta$ is surjective.*

Proof. Suppose $\exists \tau \in \mathbb{B}_{\mathcal{A}} : \nexists \beta \in \mathbb{B}$ with $\tau = \mathcal{A}\beta$. Note that for the considered type of matrices, any $\tau \in \mathbb{B}_{\mathcal{A}}$ has $\tau_i = (\mathcal{A}\beta)_i$ for some $\beta \in \mathbb{B}$, $i = 1, \dots, \bar{m}-1$. Then it must be that $\tau_{\bar{m}} \neq (\mathcal{A}\beta)_{\bar{m}}$ and $\tau_{\bar{m}} \neq 0$, implying $(\mathcal{A}\ell)_{\bar{m}} < \tau_{\bar{m}} < (\mathcal{A}u)_{\bar{m}}$ for some ℓ and u in $\mathbb{B}$. So construct $\bar{\beta}_i = \tau_i, i = 1, \dots, \bar{m}-1$, and $\bar{\beta}_i = \ell_i, i = \bar{m}, \dots, m$. Then for any dimension $i = \bar{m}, \dots, m$, incrementing $\bar{\beta}_i$ by one causes $(\mathcal{A}\bar{\beta})_{\bar{m}}$ to increase by one. This process can continue until $\bar{\beta}_i = u_i$ for $i = \bar{m}, \dots, m$, so that $(\mathcal{A}\bar{\beta})_{\bar{m}} = (\mathcal{A}u)_{\bar{m}}$, implying that $\tau_{\bar{m}}$ is eventually reached for some $\beta \in \mathbb{B}$, and thereby contradicting the existence of $\tau \in \mathbb{B}_{\mathcal{A}} : \nexists \beta \in \mathbb{B}$ with $\tau = \mathcal{A}\beta$. ■

Define $\mathbb{B}_{\mathcal{A}}^j = \{\tau \in \mathbb{B}_{\mathcal{A}} : \mathcal{A}g_j \leq \tau \text{ for } g_j \in \mathbb{B}\}$.

Lemma 2 *For any $\tau \in \mathbb{B}_{\mathcal{A}} \setminus \cup_{j=1}^n \mathbb{B}_{\mathcal{A}}^j$, $z_a(\tau) = 0$.*

Lemma 3 *For any $\tau \in \cup_{j=1}^n \mathbb{B}_{\mathcal{A}}^j$, $z_a(\tau) = \max_{j \in \{1, \dots, n\}} \{\gamma_j + z_a(\tau - \mathcal{A}g_j)\}$.*

Like Lemma 1, Lemmas 2 and 3 are used in Algorithm 1 that follows. As their proofs can be constructed using related results in [4, 8], they are omitted for brevity.

Algorithm 1 Dynamic Programming-Based Algorithm to Compute $z_a(\cdot)$

- 1: Let $\mathbb{B} = \{[0, b_1] \times [0, b_2] \times \dots \times [0, b_m]\}$, and define vector list $\mathcal{L} = \emptyset$.
 - 2: Define lower bound $\ell^0(\cdot)$, and initialize $\ell^0(\tau) = 0$ for all $\tau \in \mathbb{B}_{\mathcal{A}}$.
 - 3: **for** $j = 1, \dots, n$ **do**
 - 4: Set $\ell^0(\mathcal{A}g_j) = \max\{\ell^0(\mathcal{A}g_j), \gamma_j\}$. Insert g_j into $\mathcal{L}$, i.e., $\mathcal{L} \leftarrow \mathcal{L} \cup \{g_j\}$.
 - 5: Set $\ell^1(\tau) = \ell^0(\tau)$ for all $\tau \in \mathbb{B}_{\mathcal{A}}$, and set $k \leftarrow 1$.
 - 6: Denote the k^{th} vector in $\mathcal{L}$ by β^k and the i^{th} element of a vector β by β_i .
 - 7: Let $\beta = \beta^k$. Update all $\beta' \in \mathbb{B}$ for which $\beta' \geq \beta^k$ lexicographically as follows:
 - 8: Set $\beta_1 \leftarrow \beta_1 + 1$.
 - 9: Set $\ell^k(\mathcal{A}\beta) = \max\{\ell^k(\mathcal{A}\beta), \ell^k(\mathcal{A}\beta^k) + \ell^k(\mathcal{A}\beta - \mathcal{A}\beta^k)\}$.
 - 10: If $\beta_1 = b_1$, go to Step 11; otherwise, go to Step 8.
 - 11: If for all $i = 1, \dots, m$, $\beta_i \geq b_i$, go to Step 12. Otherwise, let $s = \min\{i : \beta_i < b_i\}$. Set $\beta_i \leftarrow \beta_i^k$ for $i = 1, \dots, s-1$. Set $\beta_s \leftarrow \beta_s + 1$ and go to Step 9.
 - 12: If $k = |\mathcal{L}|$, terminate with solution $z_a(\cdot) = \ell^k(\cdot)$. Otherwise, set $\ell^{k+1}(\mathcal{A}\beta) \leftarrow \ell^k(\mathcal{A}\beta)$ for all $\beta \in \mathbb{B}$, set $k \leftarrow k + 1$ and go to Step 7.
-

The DP-based approach described by pseudo-code in Algorithm 1 is a modification of the algorithm in [8] (see Section 4.2 in [8]) and motivated by superadditivity of $z_a(\cdot)$ (see Step 9 of Algorithm 1). We next establish correctness of the proposed approach.

Proposition 7 *Algorithm 1 terminates at iteration $k^* \leq n$ with $\ell^{k^*}(\tau) = z_a(\tau)$ for all $\tau \in \mathbb{B}_{\mathcal{A}}$.*

Proof. By Lemma 2, $z_a(\tau) = 0$ for any $\tau \in \mathbb{B}_{\mathcal{A}} \setminus \cup_{j=1}^n \mathbb{B}_{\mathcal{A}}^j$. We assume termination occurs at iteration $k^* = |\mathcal{L}|$, and so $\ell^{k^*}(\tau) = z_a(\tau)$ for any $\tau \in \mathbb{B}_{\mathcal{A}} \setminus \cup_{j=1}^n \mathbb{B}_{\mathcal{A}}^j$ because such τ are never updated in Algorithm 1 after initialization in Step 2. Over the course of Algorithm 1, updating $\ell^k(\mathcal{A}\beta)$ to $\max\{\ell^k(\mathcal{A}\beta), \ell^k(\mathcal{A}\beta^k) + \ell^k(\mathcal{A}\beta - \mathcal{A}\beta^k)\}$ ensures that at iteration k^* we have $\ell^{k^*}(\mathcal{A}\beta^k) + \ell^{k^*}(\mathcal{A}\beta - \mathcal{A}\beta^k) \leq \ell^{k^*}(\mathcal{A}\beta)$, thereby enforcing the superadditivity of $\ell^{k^*}(\cdot)$. Now suppose $\exists \tau \in \cup_{j=1}^n \mathbb{B}_{\mathcal{A}}^j : \ell^{k^*}(\tau) < z_a(\tau)$, and for all $\bar{\tau} \in \cup_{j=1}^n \mathbb{B}_{\mathcal{A}}^j : \bar{\tau} \leq \tau$ we have $\ell^{k^*}(\bar{\tau}) = z_a(\bar{\tau})$. Then $\exists j^* \in \{1, \dots, n\}$ such that $\ell^{k^*}(\tau) < \gamma_{j^*} + z_a(\tau - \mathcal{A}g_{j^*})$ by Lemma 3. Because $\ell^{k^*}(\mathcal{A}g_{j^*}) \geq \gamma_{j^*}$ and $\ell^{k^*}(\tau - \mathcal{A}g_{j^*}) = z(\tau - \mathcal{A}g_{j^*})$, it follows that $\ell^{k^*}(\mathcal{A}g_{j^*}) + \ell^{k^*}(\tau - \mathcal{A}g_{j^*}) \geq \gamma_{j^*} + z_a(\tau - \mathcal{A}g_{j^*}) > \ell^{k^*}(\tau)$, contradicting the superadditivity of $\ell^{k^*}(\cdot)$. Because the transformation defined by $\mathcal{A}$ is surjective by Lemma 1, iterating over all $\beta \in \mathbb{B}$ accounts for $z(\tau)$ for all $\tau \in \mathbb{B}_{\mathcal{A}}$. ■

Thus, Algorithm 1 is valid and its running time is $O(nm\bar{m}|\mathbb{B}|)$. While the proof of Proposition 7 follows the proof of a similar result in [8], it requires the superadditivity property established in Proposition 3. If we set $\mathcal{A}$ to the $m \times m$ identity matrix, then Algorithm 1 reduces to the DP-based algorithm from [8] with running time $O(n|\mathbb{B}|)$ (assuming that multiplications $\mathcal{A}\beta$ are not performed). One enhancement that we implement for Algorithm 1 is avoiding many of the matrix multiplications in Step 9 by a simple update from the previous value. This greatly improves the performance. Also, it should be noted that $2P$ executions of Algorithm 1 are necessary to compute value function information for both stages, prior to their subsequent exploitation in Phase 2 using (9).

Finally, we note that the proof of Proposition 7 exploits only the surjective property of the transformation defined by $\mathcal{A}$. Thus, the algorithm can compute the mapped value function for other aggregation matrices as long as this property holds. Otherwise, however, the proposed method can only be used for computation of $z_a(\tau)$ for $\tau \in \mathbb{B}_{\mathcal{A}}$ such that $\tau = \mathcal{A}\beta$ for some $\beta \in \mathbb{B}$.

3.3 Computational Experiments

We developed our algorithms in C++ and compiled the source code using g++ version 4.4.6 20110731 (Red Hat 4.4.6-3) on a Dell R610 server with 2 Intel Xeon X5690 CPUs each with 6 cores running at 3.47 GHZ and 24GB of RAM available. We used the callable library of IBM ILOG CPLEX 12.4 [7] with the default options, and implemented a time limit of 12 hours (43,200 seconds) for all algorithms.

Test Instances. Table 1 includes information on the test instances considered for our study. All test instances shared in common the values of $|\Omega| = 100$, $m = 5$, and constraint matrix density $\rho = 1$. The remaining parameters are generated the same way as in [8].

Results and Discussion. There were 30 instances generated, ten in each class having $n_1 \in \{200, 250, 300\}$ variables, with $n_2 = n_1$. The instances otherwise varied only in the maximum permissible value of b_i (either 25 or 35), as well as their maximum value for $h_i(\omega)$ (either 80 or 90). We considered seven solution methods, namely:

- The algorithm of Kong et al. [8] described in Section 1 (1);
- The method of [8], modified to use Phase 2 upper bounds based on the aggregate value functions pre-computed in Phase 1 (see (5) and related discussion) with $\bar{m} = 3$ and

				Algorithmic Runtime (in seconds)				
					$\bar{m} = 3$		$\bar{m} = 4$	
Instance	$n_1 n_2$	$b_i \in$	$h_i(\omega) \in$	CPLEX xform	Phase 1	Total	Phase 1	Total
TI-1	200	[5,35]	[50,80]	46	5,811	17,329	11,107	21,569
TI-2	200	[5,35]	[50,80]	>43,200	7,957	30,179	13,662	28,256
TI-3	200	[5,25]	[50,80]	>43,200	7,211	36,071	13,169	32,126
TI-4	200	[5,35]	[50,90]	>43,200	3,644	12,923	6,876	12,552
TI-5	200	[5,25]	[50,80]	88	6,516	14,463	13,529	19,664
TI-6	200	[5,25]	[50,90]	>43,200	>43,200		>43,200	
TI-7	200	[5,25]	[50,90]	>43,200	9,871	31,710	20,028	38,907
TI-8	200	[5,25]	[50,90]	3	397	531	729	869
TI-9	200	[5,25]	[50,90]	Memory	>43,200		>43,200	
TI-10	200	[5,35]	[50,90]	15	>43,200		>43,200	
TI-11	250	[5,25]	[50,90]	>43,200	>43,200		>43,200	
TI-12	250	[5,35]	[50,90]	19	10,031	19,463	21,944	28,258
TI-13	250	[5,25]	[50,80]	>43,200	532	8,750	1,026	8,982
TI-14	250	[5,25]	[50,90]	>43,200	>43,200		>43,200	
TI-15	250	[5,35]	[50,80]	>43,200	>43,200		>43,200	
TI-16	250	[5,25]	[50,90]	2	554	582	1,134	1,158
TI-17	250	[5,35]	[50,80]	>43,200	>43,200		>43,200	
TI-18	250	[5,35]	[50,90]	>43,200	>43,200		11,021	36,232
TI-19	250	[5,25]	[50,80]	>43,200	>43,200		>43,200	
TI-20	250	[5,25]	[50,80]	>43,200	11,038	35,245	19,834	33,234
TI-21	300	[5,25]	[50,90]	>43,200	>43,200		>43,200	
TI-22	300	[5,35]	[50,90]	13	8,946	12,860	18,523	22,410
TI-23	300	[5,25]	[50,80]	>43,200	7,033	35,593	14,012	36,005
TI-24	300	[5,35]	[50,90]	>43,200	14,802	41,663	25,630	42,226
TI-25	300	[5,25]	[50,80]	6	4,019	5,078	7,714	8,659
TI-26	300	[5,35]	[50,80]	58	259	5,036	480	5,212
TI-27	300	[5,35]	[50,80]	19	304	349	572	622
TI-28	300	[5,25]	[50,90]	4	5,515	7,910	10,113	12,506
TI-29	300	[5,25]	[50,80]	>43,200	5,005	10,145	9,640	14,473
TI-30	300	[5,25]	[50,90]	>43,200	6,861	15,652	14,134	22,145

Table 1: Performance of three competitive approaches, with best results indicated in bold.

$\bar{m} = 4$ **(2)**;

- The algorithm of Trapp et al. [12], which is also based on a value function reformulation of (SIP) and considers the same class of problems as in [8] **(1)**;
- CPLEX applied to the extensive form of (SIP) **(1)**; and
- Algorithm 1 using mapped value functions with $\bar{m} = 3$ and $\bar{m} = 4$ **(2)**.

We consider the type of aggregation matrices described in Section 3.2 and compute the corresponding value functions for all possible constraint combinations. Thus, $P = \binom{5}{3} = 10$

and $P = \binom{5}{4} = 5$ for $\bar{m} = 3$ and $\bar{m} = 4$, respectively.

In our computational experiments only the last three approaches turned out to be competitive (see results in Table 1). The first four methods failed with all of our test instances. In particular, the algorithm of Kong et al. [8] failed during Phase 1 due to relatively large magnitudes of $h_i(\omega)$, which resulted in excessive memory storage requirements. The second and third approaches based on upper bounds obtained by straightforward constraint aggregation (with $\bar{m} = 3$ and $\bar{m} = 4$, respectively) as in (5) failed to prove optimality during Phase 2. The B&B method was not able to converge to an optimal solution within the allotted time limit due to poor upper bounds, which is not surprising given the results of Proposition 5 (recall also Example 1).

The algorithm of Trapp et al. [12] focuses on the same class of problems as in [8]. It exploits level-set based ideas and computes value functions only for a select subset of right-hand sides in both stages. The results reported in [12] indicate that the algorithm can be very competitive with respect to the approach from [8], at least for some classes of problems (in particular, [12] handles instances with a larger number of constraints and similar magnitude of right-hand side values, although it has some limitations with respect to certain constraint matrices; we refer to the results and discussion in [12]). However, a relatively large magnitude of the right-hand side values present in the current study posed significant challenges to this algorithm as well, and it was not able to compute all necessary value function information within the specified time limits.

Viewed jointly, for 8 out of the 30 original instances no algorithm was able to identify a globally optimal solution. The remaining 22 instances for which at least one algorithm was able to find a globally optimal solution are detailed in Table 1. Runtimes for algorithms are reported, and when **bolded** indicate the fastest approach. For our methods we report both the time necessary for execution of Phase 1 (i.e., computation and storage of $2P$ value functions) and the total running time. It is interesting to note that both CPLEX and our approaches each had the distinct fastest times for exactly 11 of these 22 instances.

CPLEX solving the extensive form was able to solve 11 out of the 30 instances to optimality. This is where CPLEX solving the extensive form excelled: for every one of these instances, it identified the global optimal solution very quickly, in typically under a minute of runtime (average time was 25.6 seconds). For 18 of the 19 other instances, CPLEX failed to identify a globally optimal solution in the allotted time, and on one instance (TI-9) CPLEX simply exceeded memory limitations. Thus, when CPLEX solving the extensive form was able to identify a globally optimal solution, it did so in short order.

On the other hand, our mapped aggregate value function for $\bar{m} = 4$ identified a globally optimal solution within time and memory allotments in 21 out of the 30 instances, while $\bar{m} = 3$ performed almost as well, doing so in 20 of the 30 instances. The runtimes for our approach averaged 16,364.2 seconds for $\bar{m} = 3$, and 19,598.4 for $\bar{m} = 4$. While these times were significantly longer as compared to CPLEX solving the extensive form, they translate to roughly 5-6 hours of computational time, which we view as quite reasonable given the difficult nature of solving the considered class of SIPs.

When comparing our approaches to CPLEX solving the extensive form, the main reason many of the runs did not solve to optimality was due to time limitations. Interestingly, when factoring in the full 43,200 seconds as total computational time used over all 30 instances, CPLEX solving the extensive form averaged 25,094 seconds, our mapped aggregation ap-

proach with $\bar{m} = 3$ averaged 24,750 seconds, and with $\bar{m} = 4$ averaged 26,236 seconds. This shows that our approaches used no more total computational time than did CPLEX in proving global optimality, yet in nine instances our approaches identified a globally optimal solution where CPLEX could not do so within the time limit.

Thus, these limited tests appear to indicate that our approach is able to solve almost half as many more instances to global optimality as compared to CPLEX solving the extensive form. This came at the expense of longer, but still rather reasonable runtimes (at least in our view), likely due to the up-front cost of computing and storing the mapped value functions. Additionally, our approach was also able to solve all but one (TI-10) of the eleven instances that CPLEX could solve (again, though, with longer runtimes).

There exists an interesting trade-off with respect to the running times of our approach with $\bar{m} = 3$ and $\bar{m} = 4$. In particular, the value function computations in Phase 1 are substantially easier for the smaller value of $\bar{m} = 3$ than for $\bar{m} = 4$ (see running times of Phase 1 in Table 1), which is intuitive due the substantial difference in sizes of the corresponding $\mathbb{B}_A$ sets. However, the quality of upper bounds decreases when more constraints are aggregated. On the other hand, the improved upper bound quality for $\bar{m} = 4$ often allows the B&B algorithm to complete Phase 2 reasonably faster, which result in the improvement of the total running time for some test instances.

4. Concluding Remarks

We use constraint aggregation concepts to introduce a simple approach to mitigate some of the intensive memory requirements for value function storage in the context of solving two-stage SIPs. The approach highlights one possible way towards alleviating the memory storage issues found in some methods in the literature [8, 10]. In our experiments we use a set of test instances with similar parameters to [8] but with larger right-hand sides. Limited computational experiments with a rather naïve implementation of the proposed approach on a class of test instances still compare favorably to the algorithm from [8], another related method in [12], and CPLEX.

Our treatment leaves a number of issues for future investigation. First, there exists an interesting trade-off between the quality of the obtained upper bounds and the up-front costs of computing mapped value functions as the number of aggregated constraints changes. Second, one can consider methods that combine the constraint aggregation concept with level-set based ideas from [12]. Third, it would be interesting to study applicability of the proposed approach in the context of stochastic nonlinear IPs, e.g., the quadratic form in [10]. In summary, we believe that constraint aggregation ideas may be useful, at least for certain problem classes, in the context of value functions and stochastic integer optimization.

Acknowledgements. The authors thank two anonymous referees whose constructive comments substantially improved this manuscript.

References

- [1] S. Ahmed, M. Tawarmalani, and N. Sahinidis. A finite branch and bound algorithm for two-stage stochastic integer programs. *Mathematical Programming*, 100(2):355–377, 2004.
- [2] C. Carøe and R. Schultz. Dual decomposition in stochastic integer programming. *Operations Research Letters*, 24(1):37–45, 1999.
- [3] C.C. Carøe and J. Tind. L-shaped decomposition of two-stage stochastic programs with integer recourse. *Mathematical Programming*, 83(3):451–464, 1998.
- [4] P.C. Gilmore and R.E. Gomory. The theory and computation of knapsack functions. *Operations Research*, 14(6):1045–1074, 1966.
- [5] F. Glover. Surrogate constraint duality in mathematical programming. *Operations Research*, 23(3):434–451, 1975.
- [6] R. Hemmecke and R. Schultz. Decomposition of test sets in stochastic integer programming. *Mathematical Programming*, 94(2):323–341, 2003.
- [7] IBM. *IBM ILOG CPLEX 12.4 User’s Manual*. IBM ILOG CPLEX Division, Incline Village, NV, 2012.
- [8] N. Kong, A.J. Schaefer, and B. Hunsaker. Two-stage integer programs with stochastic right-hand sides: A superadditive dual approach. *Mathematical Programming*, 108(2):275–296, 2006.
- [9] G.L. Nemhauser and L.A. Wolsey. *Integer and Combinatorial Optimization*. John Wiley & Sons, Inc., 1988.
- [10] O.Y. Özaltın, O.A. Prokopyev, and A.J. Schaefer. Two-stage quadratic integer programs with stochastic right-hand sides. *Mathematical Programming*, 133(1/2):121–158, 2012.
- [11] R. Schultz. On structure and stability in stochastic programs with random technology matrix and complete integer recourse. *Mathematical Programming*, 70(1):73–90, 1995.
- [12] A.C. Trapp, O.A. Prokopyev, and A.J. Schaefer. On a level-set characterization of the value function of an integer program and its application to stochastic programming. *Operations Research*, 61(2):498–511, 2013.